PRACTICAL 1 — DI & Injection

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
     xsi:schemaLocation="
     http://maven.apache.org/POM/4.0.0
     https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>SpringDIProject</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <dependencies>

     <dependency>
       <groupId>org.springframework</groupId>
       <artifactId>spring-context</artifactId>
       <version>6.1.8</version>
     </dependency>

  </dependencies>

</project>
```

Main.java

```
package org.example;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {

  public static void main(String[] args) {

     ApplicationContext context =
          new AnnotationConfigApplicationContext(
               AppConfig.class
          );

     MessageController controller =
          context.getBean(
               MessageController.class
          );

     controller.displayMessage();
  }
}
```

MessageController.java

```
package org.example;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
public class MessageController {

    @Autowired
    private MessageService messageService;

    public void displayMessage() {

        System.out.println(
             messageService.getMessage()
        );
    }
}
```

MessageService.java

```
package org.example;

import org.springframework.stereotype.Component;

@Component
public class MessageService {

    public String getMessage() {
```

```
        return "Hello from FIRST PROJECT Service";
    }
}
```

AppConfig.java

```
package org.example;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("org.example")
public class AppConfig {

}
```

PRACTICAL 1 - OUTPUT

```
Hello from FIRST PROJECT Service
```

PRACTICAL 2 — Bean Factory and XML Configuration

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
     https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>SpringXMLDemo</artifactId>
    <version>1.0</version>

    <dependencies>
      <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>6.2.6</version>
      </dependency>
    </dependencies>

</project>
```

EmailService.java

```
package com.example;

public class EmailService {

    public void sendEmail() {
      System.out.println("Email Sent Successfully.");
    }
}
```

CustomerService.java

```
package com.example;

public class CustomerService {

    private EmailService emailService;

    public void setEmailService(EmailService emailService) {
      this.emailService = emailService;
    }

    public void registerCustomer() {
      System.out.println("Customer Registered Successfully.");
      emailService.sendEmail();
    }
}
```

beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="
   http://www.springframework.org/schema/beans
   https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="emailService"
       class="com.example.EmailService"/>

    <bean id="customerService"
       class="com.example.CustomerService">

      <property name="emailService"
           ref="emailService"/>

    </bean>

</beans>
```

Main.java

```
package com.example;

import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {

    public static void main(String[] args) {

        ClassPathXmlApplicationContext context =
```

```
            new ClassPathXmlApplicationContext("beans.xml");

        CustomerService customer =
            context.getBean("customerService", CustomerService.class);

        customer.registerCustomer();

        context.close();
    }
}
```

PRACTICAL 2 - OUTPUT

```
Customer Registered Successfully.
Email Sent Successfully.
```

PRACTICAL 3 — Component-Based Application Development

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
     https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>ComponentDemo</artifactId>
    <version>1.0</version>

    <dependencies>
      <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>6.2.6</version>
      </dependency>
    </dependencies>

</project>
```

StudentService.java

```
package org.example.service;

import org.springframework.stereotype.Service;

@Service
public class StudentService {

    public String getStudentDetails() {

        return """
             Student ID : 101
             Student Name : Rahul
             Course : MCA
             Semester : 3
             """;
    }
}
```

StudentController.java

```
package org.example.controller;

import org.example.service.StudentService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;

@Controller
public class StudentController {

    @Autowired
    StudentService studentService;

    public void displayStudent() {

        System.out.println(
             studentService.getStudentDetails()
        );
    }
}
```

AppConfig.java

```
package org.example.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages = "org.example")
public class AppConfig {

}
```

Main.java

```
package org.example;

import org.example.config.AppConfig;
import org.example.controller.StudentController;
import org.springframework.context.ApplicationContext;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {

    public static void main(String[] args) {

        ApplicationContext context =
             new AnnotationConfigApplicationContext(AppConfig.class);

        StudentController controller =
            context.getBean(StudentController.class);

        controller.displayStudent();
    }
}
```

PRACTICAL 3 - OUTPUT

```
Student ID : 101
Student Name : Rahul
Course : MCA
Semester : 3
```

PRACTICAL 4 — Model-View-Controller (MVC) Architecture

WebConfig.java

```
package org.example.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.ViewResolverRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "org.example")
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void configureViewResolvers(ViewResolverRegistry registry) {
      registry.jsp("/WEB-INF/views/", ".jsp");
    }
}
```

HomeController.java

```
package org.example.controller;

import org.example.model.Student;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

@Controller
public class HomeController {

    @GetMapping("/")
    public String home() {
      return "studentForm";
    }

    @GetMapping("/studentForm")
    public String studentForm() {
      return "studentForm";
    }

    @PostMapping("/register")
    public String register(
         @RequestParam("name") String name,
         @RequestParam("course") String course,
         Model model) {

        Student student = new Student();

        student.setName(name);
        student.setCourse(course);

        model.addAttribute("student", student);

        return "result";
    }
}
```

Student.java

```
package org.example.model;

public class Student {

    private String name;
    private String course;

    public Student() {
    }

    public String getName() {
      return name;
    }

    public void setName(String name) {
      this.name = name;
    }

    public String getCourse() {
      return course;
    }

    public void setCourse(String course) {
```

```
      this.course = course;
    }
}
```

studentForm.jsp

```
<%@ page contentType="text/html;charset=UTF-8" %>

<html>
<head>
  <title>Student Registration</title>
</head>

<body>

    <h2>Student Registration Form</h2>

    <form action="register" method="post">

      Name:
      <input type="text" name="name" required>
      <br><br>

      Course:
      <input type="text" name="course" required>
      <br><br>

      <input type="submit" value="Register">

  </form>

</body>
</html>
```

result.jsp

```
<%@ page contentType="text/html;charset=UTF-8" %>

<html>
<head>
  <title>Student Details</title>
</head>

<body>

  <h2>Student Details</h2>

  <p>Name: ${student.name}</p>

  <p>Course: ${student.course}</p>

</body>
</html>
```

PRACTICAL 4 - OUTPUT

Student Details

Name: Rahul
Course: MCA

PRACTICAL 5 — Reading HTML Form Data

WebAppInitializer.java

```
package org.example.config;

import org.springframework.web.servlet.support.AbstractAnnotationConfigDispatcherServletInitializer;

public class WebAppInitializer
     extends AbstractAnnotationConfigDispatcherServletInitializer {

    @Override
    protected Class<?>[] getRootConfigClasses() {
      return null;
    }

    @Override
    protected Class<?>[] getServletConfigClasses() {
      return new Class<?>[]{
           WebConfig.class
      };
    }

    @Override
    protected String[] getServletMappings() {
      return new String[]{
           "/"
      };
    }
}
```

WebConfig.java

```
package org.example.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.ViewResolverRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
@EnableWebMvc
@ComponentScan("org.example")
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void configureViewResolvers(ViewResolverRegistry registry) {

        registry.jsp(
             "/WEB-INF/views/",
             ".jsp"
        );
    }
}
```

Student.java

```
package org.example.model;

public class Student {

    private String name;
    private String email;
    private String course;

    public String getName() {
      return name;
    }

    public void setName(String name) {
      this.name = name;
    }

    public String getEmail() {
      return email;
    }

    public void setEmail(String email) {
      this.email = email;
    }

    public String getCourse() {
      return course;
    }

    public void setCourse(String course) {
      this.course = course;
    }
```

```
}
```

StudentController.java

```
package org.example.controller;

import org.example.model.Student;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class StudentController {

    @GetMapping("/")
    public String home() {
      return "redirect:/studentForm";
    }

    @GetMapping("/studentForm")
    public String showForm() {
      return "studentForm";
    }

    @PostMapping("/register")
    public String registerStudent(Student student, Model model) {

        model.addAttribute("student", student);

        return "result";
    }
}
```

studentForm.jsp

```
<html>
<head>
  <title>Student Form</title>
</head>

<body>

<h2>Student Registration Form</h2>

<form action="register" method="post">

  Name:
  <input type="text" name="name">
  <br><br>

  Email:
  <input type="email" name="email">
  <br><br>

  Course:
  <input type="text" name="course">
  <br><br>

  <input type="submit" value="Register">

</form>

</body>
</html>
```

result.jsp

```
<html>
<head>
  <title>Student Details</title>
</head>

<body>

<h2>Student Details</h2>

Name: ${student.name}
<br><br>

Email: ${student.email}
<br><br>

Course: ${student.course}

</body>
</html>
```

PRACTICAL 5 - OUTPUT

Student Details

Name: Rahul
Email: rahul@gmail.com
Course: MCA

PRACTICAL 6 — Validation and Custom Validation

WebAppInitializer.java

```
package org.example.config;

import org.springframework.web.servlet.support.AbstractAnnotationConfigDispatcherServletInitializer;

public class WebAppInitializer
        extends AbstractAnnotationConfigDispatcherServletInitializer {

    @Override
    protected Class<?>[] getRootConfigClasses() {
        return null;
    }

    @Override
    protected Class<?>[] getServletConfigClasses() {
        return new Class<?>[]{
             WebConfig.class
        };
    }

    @Override
    protected String[] getServletMappings() {
        return new String[]{
             "/"
        };
    }
}
```

WebConfig.java

```
package org.example.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.ViewResolverRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
@EnableWebMvc
@ComponentScan("org.example")
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void configureViewResolvers(
             ViewResolverRegistry registry) {

        registry.jsp(
               "/WEB-INF/views/",
               ".jsp"
        );
    }
}
```

Student.java

```
package org.example.model;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

public class Student {

  @NotBlank(message = "Name is required")
  private String name;

  @Email(message = "Enter valid email")
  private String email;

  @Size(
       min = 6,
       max = 12,
       message = "Password must be 6-12 characters"
  )
  private String password;

  @Pattern(
       regexp = "[0-9]{10}",
       message = "Mobile should contain 10 digits"
  )
  private String mobile;

  private String course;
```

```
  public String getName() {
    return name;
  }

  public void setName(String name) {
    this.name = name;
  }

  public String getEmail() {
    return email;
  }

  public void setEmail(String email) {
    this.email = email;
  }

  public String getPassword() {
    return password;
  }

  public void setPassword(String password) {
    this.password = password;
  }

  public String getMobile() {
    return mobile;
  }

  public void setMobile(String mobile) {
    this.mobile = mobile;
  }

  public String getCourse() {
    return course;
  }

  public void setCourse(String course) {
    this.course = course;
  }
}
```

StudentValidator.java

```
package org.example.validator;

import org.example.model.Student;

import org.springframework.stereotype.Component;
import org.springframework.validation.Errors;
import org.springframework.validation.Validator;

@Component
public class StudentValidator implements Validator {

  @Override
  public boolean supports(Class<?> clazz) {
      return Student.class.equals(clazz);
  }

  @Override
  public void validate(
        Object target,
        Errors errors) {

      Student student = (Student) target;

      String course = student.getCourse();

      if (course == null ||
           !(course.equalsIgnoreCase("BCA") ||
             course.equalsIgnoreCase("MCA") ||
             course.equalsIgnoreCase("B.Tech"))) {

        errors.rejectValue(
                  "course",
                  "course.invalid",
                  "Only BCA, MCA and B.Tech are allowed."
              );
        }
    }
}
```

StudentController.java

```
package org.example.controller;

import jakarta.validation.Valid;

import org.example.model.Student;
import org.example.validator.StudentValidator;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class StudentController {

@Autowired
private StudentValidator validator;

@GetMapping("/")
public String home() {
    return "redirect:/studentForm";
}

@GetMapping("/studentForm")
public String showForm() {
    return "studentForm";
}

@PostMapping("/register")
public String submitForm(
      @Valid
      @ModelAttribute("student")
      Student student,
      BindingResult result) {

    validator.validate(student, result);

    if (result.hasErrors()) {
            return "studentForm";
    }

    return "success";
}
}
```

studentForm.jsp

```
<%@ taglib prefix="form"
    uri="http://www.springframework.org/tags/form" %>

<html>

<head>
    <title>Student Registration</title>
</head>

<body>

<h2>Student Registration Form</h2>

<form:form action="register"
            method="post"
            modelAttribute="student">
Name:
<form:input path="name"/>
<form:errors path="name"/>
<br><br>

Email:
<form:input path="email"/>
<form:errors path="email"/>
<br><br>

Password:
<form:password path="password"/>
<form:errors path="password"/>
<br><br>

Mobile:
<form:input path="mobile"/>
<form:errors path="mobile"/>
<br><br>

Course:
<form:input path="course"/>
<form:errors path="course"/>
<br><br>

  <input type="submit"
         value="Register"/>

</form:form>

</body>
</html>
```

success.jsp

```
<html>

<head>
  <title>Success</title>
</head>

<body>

<h2>Registration Successful</h2>

</body>
</html>
```

PRACTICAL 6 - OUTPUT

Name is required

Enter valid email

Password must be 6-12 characters

Mobile should contain 10 digits

Only BCA, MCA and B.Tech are allowed.

PRACTICAL 7 — Aspect-Oriented Spring (AOP)

pom.xml

```
<dependencies>

    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>6.1.8</version>
    </dependency>

    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-aop</artifactId>
        <version>6.1.8</version>
    </dependency>

    <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjweaver</artifactId>
        <version>1.9.22</version>
    </dependency>

</dependencies>
```

StudentService.java

```
package com.example.service;

import org.springframework.stereotype.Component;

@Component
public class StudentService {

    public void displayMessage() {

        System.out.println("Welcome to Spring AOP Practical");

    }

    public void calculate() {

        int sum = 10 + 20;

        System.out.println("Addition = " + sum);

    }
}
```

LoggingAspect.java

```
package com.example.aspect;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.After;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.example.service.StudentService.*(..))")
    public void beforeMethod(JoinPoint jp) {

        System.out.println("--------------------------------");
        System.out.println("LOG : Method Started");
        System.out.println("Method Name : " + jp.getSignature().getName());
        System.out.println("--------------------------------");

    }

    @After("execution(* com.example.service.StudentService.*(..))")
    public void afterMethod(JoinPoint jp) {

        System.out.println("--------------------------------");
        System.out.println("LOG : Method Completed");
        System.out.println("Method Name : " + jp.getSignature().getName());
        System.out.println("--------------------------------");

    }

}
```

AppConfig.java

```
package com.example.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableAspectJAutoProxy;

@Configuration
@ComponentScan(basePackages = "com.example")
@EnableAspectJAutoProxy
public class AppConfig {

}
```

MainApp.java

```
package com.example;

import com.example.config.AppConfig;
import com.example.service.StudentService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class MainApp {

    public static void main(String[] args) {

        ApplicationContext context =
                new AnnotationConfigApplicationContext(AppConfig.class);

        StudentService service =
                context.getBean(StudentService.class);

        service.displayMessage();

        System.out.println();

        service.calculate();

    }

}
```

PRACTICAL 7 - OUTPUT

```
--------------------------------
LOG : Method Started
Method Name : displayMessage
--------------------------------
Welcome to Spring AOP Practical

--------------------------------
LOG : Method Completed
Method Name : displayMessage
--------------------------------

--------------------------------
LOG : Method Started
Method Name : calculate
--------------------------------

Addition = 30

--------------------------------
LOG : Method Completed
Method Name : calculate
--------------------------------
```

PRACTICAL 8 — Spring JDBC Abstraction Layer

database.sql

```
CREATE DATABASE college;

USE college;

CREATE TABLE student (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100),
    course VARCHAR(100)
);
```

Student.java

```
package com.example;

public class Student {

    private int id;
    private String name;
    private String email;
    private String course;

    public Student() {
    }

    public Student(int id, String name, String email, String course) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.course = course;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }

    @Override
    public String toString() {
        return id + " " + name + " " + email + " " + course;
    }
}
```

StudentDAO.java

```
package com.example;

import java.util.List;

public interface StudentDAO {

    void saveStudent(Student student);

    List<Student> getAllStudents();

    void updateStudent(Student student);

    void deleteStudent(int id);
```

```
}
```

StudentDAOImpl.java

```
package com.example;

import org.springframework.jdbc.core.JdbcTemplate;

import java.util.List;

public class StudentDAOImpl implements StudentDAO {

    private JdbcTemplate jdbcTemplate;

    public void setJdbcTemplate(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    @Override
    public void saveStudent(Student student) {

        String sql = "INSERT INTO student(name,email,course) VALUES(?,?,?)";

        jdbcTemplate.update(
                sql,
                student.getName(),
                student.getEmail(),
                student.getCourse()
        );
    }

    @Override
    public List<Student> getAllStudents() {

        String sql = "SELECT * FROM student";

        return jdbcTemplate.query(sql, (rs, rowNum) ->
                new Student(
                        rs.getInt("id"),
                        rs.getString("name"),
                        rs.getString("email"),
                        rs.getString("course")
                )
        );
    }

    @Override
    public void updateStudent(Student student) {

        String sql = "UPDATE student SET name=?, email=?, course=? WHERE id=?";

        jdbcTemplate.update(
                sql,
                student.getName(),
                student.getEmail(),
                student.getCourse(),
                student.getId()
        );
    }

    @Override
    public void deleteStudent(int id) {

        String sql = "DELETE FROM student WHERE id=?";

        jdbcTemplate.update(sql, id);
    }
}
```

AppConfig.java

```
package com.example;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;

import javax.sql.DataSource;

@Configuration
public class AppConfig {

    @Bean
    public DataSource dataSource() {

        DriverManagerDataSource ds = new DriverManagerDataSource();

        ds.setDriverClassName("com.mysql.cj.jdbc.Driver");
        ds.setUrl("jdbc:mysql://localhost:3306/college");
        ds.setUsername("root");
```

```
        ds.setPassword("root");

        return ds;
    }

    @Bean
    public JdbcTemplate jdbcTemplate() {
        return new JdbcTemplate(dataSource());
    }

    @Bean
    public StudentDAOImpl studentDAO() {

        StudentDAOImpl dao = new StudentDAOImpl();
        dao.setJdbcTemplate(jdbcTemplate());

        return dao;
    }
}
```

Main.java

```
package com.example;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

import java.util.List;

public class Main {

    public static void main(String[] args) {

        ApplicationContext context =
                new AnnotationConfigApplicationContext(AppConfig.class);

        StudentDAO dao = context.getBean(StudentDAO.class);

        // CREATE
        Student student =
                new Student(0, "Rahul", "rahul@gmail.com", "MCA");

        dao.saveStudent(student);

        System.out.println("Student inserted successfully.");

        // READ
        System.out.println("\nStudent Records:");

        List<Student> students = dao.getAllStudents();

        for (Student s : students) {
            System.out.println(s);
        }

        // UPDATE
        Student update =
                new Student(1, "Rahul Kumar",
                        "rahulkumar@gmail.com", "MCA");

        dao.updateStudent(update);

        System.out.println("\nStudent updated successfully.");

        // DELETE
        dao.deleteStudent(2);

        System.out.println("Student deleted successfully.");
    }
}
```

PRACTICAL 8 - OUTPUT

```
Student inserted successfully.

Student Records:
1 Rahul rahul@gmail.com MCA
2 Priya priya@gmail.com BCA

Student updated successfully.
Student deleted successfully.
```

PRACTICAL 9 — Integrating Hibernate with Spring

database.sql

```
CREATE DATABASE college;

USE college;

CREATE TABLE student (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100),
    course VARCHAR(100)
);
```

Student.java

```
package com.example;

import jakarta.persistence.*;

@Entity
@Table(name = "student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String name;
    private String email;
    private String course;

    public Student() {
    }

    public Student(String name, String email, String course) {
        this.name = name;
        this.email = email;
        this.course = course;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getCourse() {
        return course;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public void setCourse(String course) {
        this.course = course;
    }

    @Override
    public String toString() {
        return id + " " + name + " " + email + " " + course;
    }
}
```

StudentDAO.java

```
package com.example;

import java.util.List;

public interface StudentDAO {

    void saveStudent(Student student);

    List<Student> getStudents();
}
```

StudentDAOImpl.java

```
package com.example;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

public class StudentDAOImpl implements StudentDAO {

    private SessionFactory sessionFactory;

    public void setSessionFactory(SessionFactory sessionFactory) {
        this.sessionFactory = sessionFactory;
    }

    @Override
    @Transactional
    public void saveStudent(Student student) {

        Session session = sessionFactory.getCurrentSession();

        session.persist(student);
    }

    @Override
    @Transactional(readOnly = true)
    public List<Student> getStudents() {

        Session session = sessionFactory.getCurrentSession();

        return session
                .createQuery("from Student", Student.class)
                .getResultList();
    }
}
```

AppConfig.java

```
package com.example;

import org.hibernate.SessionFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.orm.hibernate5.HibernateTransactionManager;
import org.springframework.orm.hibernate5.LocalSessionFactoryBean;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import java.util.Properties;

@Configuration
@EnableTransactionManagement
public class AppConfig {

    @Bean
    public DataSource dataSource() {

        DriverManagerDataSource ds =
                new DriverManagerDataSource();

        ds.setDriverClassName("com.mysql.cj.jdbc.Driver");
        ds.setUrl("jdbc:mysql://localhost:3306/college");
        ds.setUsername("root");
        ds.setPassword("root");

        return ds;
    }

    @Bean
    public LocalSessionFactoryBean sessionFactory() {

        LocalSessionFactoryBean factory =
                new LocalSessionFactoryBean();

        factory.setDataSource(dataSource());
        factory.setPackagesToScan("com.example");

        Properties properties = new Properties();

        properties.setProperty(
                "hibernate.dialect",
                "org.hibernate.dialect.MySQLDialect"
        );

        properties.setProperty(
                "hibernate.hbm2ddl.auto",
                "update"
        );
```

```
        properties.setProperty(
                "hibernate.show_sql",
                "true"
        );

        factory.setHibernateProperties(properties);

        return factory;
    }

    @Bean
    public HibernateTransactionManager transactionManager(
            SessionFactory sessionFactory) {

        return new HibernateTransactionManager(sessionFactory);
    }

    @Bean
    public StudentDAOImpl studentDAO(
            SessionFactory sessionFactory) {

        StudentDAOImpl dao = new StudentDAOImpl();

        dao.setSessionFactory(sessionFactory);

        return dao;
    }
}
```

Main.java

```
package com.example;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {

    public static void main(String[] args) {

        ApplicationContext context =
                new AnnotationConfigApplicationContext(AppConfig.class);

        StudentDAO dao = context.getBean(StudentDAO.class);

        Student student = new Student(
                "Amit",
                "amit@gmail.com",
                "MCA"
        );

        dao.saveStudent(student);

        System.out.println("Student saved successfully.");

        System.out.println("\nStudent Records:");

        for (Student s : dao.getStudents()) {
            System.out.println(s);
        }
    }
}
```

PRACTICAL 9 - OUTPUT

```
Hibernate:
    insert into student
    (course,email,name)
    values
    (?,?,?)

Student saved successfully.

Student Records:
1 Amit amit@gmail.com MCA
2 Rahul rahul@gmail.com MCA
```

PRACTICAL 10 — Securing Spring: Enabling Spring Security

SecurityConfig.java

```
package com.example;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public InMemoryUserDetailsManager users() {

        UserDetails user = User.withUsername("student")
                .password("{noop}1234")
                .roles("USER")
                .build();

        UserDetails admin = User.withUsername("admin")
                .password("{noop}admin123")
                .roles("ADMIN")
                .build();

        return new InMemoryUserDetailsManager(user, admin);
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {

        http.authorizeHttpRequests(auth -> auth
                .requestMatchers("/home").permitAll()
                .requestMatchers("/user").hasRole("USER")
                .requestMatchers("/admin").hasRole("ADMIN")
                .anyRequest().authenticated()
        );

        http.formLogin(form -> form.permitAll());

        return http.build();
    }
}
```

HomeController.java

```
package com.example;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @GetMapping("/home")
    public String home() {
        return "home";
    }

    @GetMapping("/user")
    public String user() {
        return "user";
    }

    @GetMapping("/admin")
    public String admin() {
        return "admin";
    }
}
```

home.jsp

```
<html>
<body>

<h1>Welcome to Home Page</h1>

<a href="user">User Page</a>
<br>
<a href="admin">Admin Page</a>

</body>
</html>
```

user.jsp

```
<html>
<body>

<h1>Welcome User</h1>

<p>You are authorized as USER.</p>

</body>
</html>
```

admin.jsp

```
<html>
<body>

<h1>Welcome Admin</h1>

<p>You are authorized as ADMIN.</p>

</body>
</html>
```

PRACTICAL 10 - OUTPUT

```
Welcome User

You are authorized as USER.

Welcome Admin

You are authorized as ADMIN.

Normal user trying to access /admin:
Access is denied.
```

PRACTICAL 11 — Spring REST: Overview of REST Web Service

Student.java

```
package com.example;

public class Student {

    private int id;
    private String name;
    private String course;

    public Student() {
    }

    public Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

StudentController.java

```
package com.example;

import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {

    private List<Student> students = new ArrayList<>();

    public StudentController() {

        students.add(
                new Student(1, "Rahul", "BCA")
        );

        students.add(
                new Student(2, "Priya", "MCA")
        );
    }

    // GET
    @GetMapping
    public List<Student> getStudents() {
        return students;
    }

    // GET by ID
    @GetMapping("/{id}")
    public Student getStudent(@PathVariable int id) {

        for (Student s : students) {

            if (s.getId() == id) {
                return s;
            }
        }

        return null;
```

```
    }

    // POST
    @PostMapping
    public Student addStudent(
            @RequestBody Student student) {

        students.add(student);

        return student;
    }

    // PUT
    @PutMapping("/{id}")
    public Student updateStudent(
            @PathVariable int id,
            @RequestBody Student student) {

        for (Student s : students) {

            if (s.getId() == id) {

                s.setName(student.getName());
                s.setCourse(student.getCourse());

                return s;
            }
        }

        return null;
    }

    // DELETE
    @DeleteMapping("/{id}")
    public String deleteStudent(
            @PathVariable int id) {

        students.removeIf(
                student -> student.getId() == id
        );

        return "Student deleted successfully";
    }
}
```

PRACTICAL 11 - OUTPUT

GET


```
[
  {
    "id": 1,
    "name": "Rahul",
    "course": "BCA"
  },
  {
    "id": 2,
    "name": "Priya",
    "course": "MCA"
  }
]
```


POST


```
{
  "id": 3,
  "name": "Amit",
  "course": "MCA"
}
```


PUT


```
{
  "id": 3,
  "name": "Amit Kumar",
  "course": "MCA"
}
```


DELETE

```
Student deleted successfully
```

PRACTICAL 12 — JSON Data Binding

Student.java

```
package com.example;

public class Student {

    private int id;
    private String name;
    private String email;
    private String course;

    public Student() {
    }

    public Student(int id, String name,
                   String email, String course) {

        this.id = id;
        this.name = name;
        this.email = email;
        this.course = course;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

StudentController.java

```
package com.example;

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/student")
public class StudentController {

    // Java Object → JSON
    @GetMapping
    public Student getStudent() {

        Student student = new Student(
                1,
                "Rahul",
                "rahul@gmail.com",
                "BCA"
        );

        return student;
    }

    // JSON → Java Object
    @PostMapping
    public String addStudent(
            @RequestBody Student student) {

        return "Student received: "
                + student.getName()
                + ", Course: "
```

```
                + student.getCourse();
    }
}
```

PRACTICAL 12 - OUTPUT

```
GET
{
    "id": 1,
    "name": "Rahul",
    "email": "rahul@gmail.com",
    "course": "BCA"
}
```

```
POST
Student received: Priya, Course: MCA
```

PRACTICAL 13 — REST CRUD Features

Student.java

```
package com.example;

public class Student {

    private int id;
    private String name;
    private String course;

    public Student() {
    }

    public Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

StudentController.java

```
package com.example;

import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {

    private List<Student> students = new ArrayList<>();

    public StudentController() {

        students.add(
                new Student(1, "Rahul", "BCA")
        );

        students.add(
                new Student(2, "Priya", "MCA")
        );
    }

    // CREATE - Add Student
    @PostMapping
    public Student createStudent(
            @RequestBody Student student) {

        students.add(student);

        return student;
    }

    // READ - Get All Students
    @GetMapping
    public List<Student> getStudents() {

        return students;
    }

    // READ - Get Student by ID
```

```
    @GetMapping("/{id}")
    public Student getStudent(
            @PathVariable int id) {

        for (Student student : students) {

            if (student.getId() == id) {
                return student;
            }
        }

        return null;
    }

    // UPDATE - Update Student
    @PutMapping("/{id}")
    public Student updateStudent(
            @PathVariable int id,
            @RequestBody Student student) {

        for (Student s : students) {

            if (s.getId() == id) {

                s.setName(student.getName());
                s.setCourse(student.getCourse());

                return s;
            }
        }

        return null;
    }

    // DELETE - Delete Student
    @DeleteMapping("/{id}")
    public String deleteStudent(
            @PathVariable int id) {

        students.removeIf(
                student -> student.getId() == id
        );

        return "Student deleted successfully";
    }
}
```

PRACTICAL 13 - OUTPUT

CREATE - POST


```
{
  "id": 3,
  "name": "Amit",
  "course": "MCA"
}
```


READ - GET


```
[
  {
    "id": 1,
    "name": "Rahul",
    "course": "BCA"
  },
  {
    "id": 2,
    "name": "Priya",
    "course": "MCA"
  },
  {
    "id": 3,
    "name": "Amit",
    "course": "MCA"
  }
]
```


UPDATE - PUT


```
{
  "id": 3,
  "name": "Amit Kumar",
  "course": "BCA"
}
```


DELETE - DELETE

```
Student deleted successfully
```

PRACTICAL 14 — Retrieve POJOs as JSON & Exception Handling

Student.java

```
package com.example;

public class Student {

    private int id;
    private String name;
    private String course;

    public Student() {
    }

    public Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

ErrorResponse.java

```
package com.example;

public class ErrorResponse {

    private String message;
    private String status;

    public ErrorResponse() {
    }

    public ErrorResponse(String message, String status) {
        this.message = message;
        this.status = status;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

StudentNotFoundException.java

```
package com.example;

public class StudentNotFoundException
        extends RuntimeException {

    public StudentNotFoundException(String message) {
        super(message);
```

```
    }
}
```

StudentController.java

```
package com.example;

import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {

    private List<Student> students = new ArrayList<>();

    public StudentController() {

        students.add(
                new Student(1, "Rahul", "BCA")
        );

        students.add(
                new Student(2, "Priya", "MCA")
        );
    }

    // Retrieve POJO as JSON
    @GetMapping("/{id}")
    public Student getStudent(
            @PathVariable int id) {

        for (Student student : students) {

            if (student.getId() == id) {
                return student;
            }
        }

        throw new StudentNotFoundException(
                "Student with ID " + id + " not found"
        );
    }

    // Exception Handler
    @ExceptionHandler(StudentNotFoundException.class)
    public ErrorResponse handleStudentNotFound(
            StudentNotFoundException exception) {

        return new ErrorResponse(
                exception.getMessage(),
                "404"
        );
    }
}
```

PRACTICAL 14 - OUTPUT

Case 1 - Student Found


```
{
  "id": 1,
  "name": "Rahul",
  "course": "BCA"
}
```


Case 2 - Student Not Found


```
{
  "message": "Student with ID 10 not found",
  "status": "404"
}
```